

Reviewer Pack — Minimal Number of Affine Subspaces in Vector Spaces and Circ...

Entry 2b2d30ba76b8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 10:26:37 UTC

Minimal Number of Affine Subspaces in Vector Spaces and Circuit Depth Inequality

Entry ID: 2b2d30ba76b8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 10:26:37 UTC

1. Conjecture

Field A (mathematical branch): Linear Algebra (Affine Geometry) **Field B** (complexity object): Boolean Circuit Complexity

Statement:

For every Boolean circuit C with depth D , there exists a finite-dimensional vector space V over the field F_2 such that the minimal number of affine subspaces in V required to represent the gates of C is $O(D^2 \log n)$, where n is the number of variables.

Rationale (proposer's reasoning):

Affine geometry has been applied to computational complexity theory, particularly in the context of representation theory. By mapping boolean circuits to vector spaces, we can leverage geometric properties to derive lower bounds on circuit complexity. This conjecture proposes a direct relationship between the number of affine subspaces and the depth of a Boolean circuit, which could reveal new insights into circuit complexity.

Taxonomy category: AffineGeometry × CircuitComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 817142fd402faba4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each Boolean circuit C with varying depths up to $D=40$, if the minimum number of affine subspaces required in vector space V over F_2 is within $O(D^2 \log n)$, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(depth, n):
        if depth == 0:
            return [random.choice([0, 1]) for _ in range(n)]
        else:
            inputs = generate_circuit(depth - 1, n)
            gate = random.choice(['AND', 'OR'])
            if gate == 'AND':
                return [a and b for a, b in zip(inputs[:n//2], inputs[n//2:])]
            else:
                return [a or b for a, b in zip(inputs[:n//2], inputs[n//2:])]

    def generate_vector_space(circuit):
        n = len(circuit)
        V = []
        for i in range(n):
            if circuit[i] == 0:
                V.append([1, 0])
            else:
                V.append([0, 1])
        return V

    def find_minimal_affine_subspaces(V):
        m, n = len(V), len(V[0])
        subspaces = []
        for i in range(m):
            subspace = [V[i]]
            for j in range(i + 1, m):
                if all(V[j][k] == V[i][k] for k in range(n)):
                    subspace.append(V[j])
            subspaces.append(subspace)
        return subspaces
```

```

def count_affine_subspaces(V):
    subspaces = find_minimal_affine_subspaces(V)
    return len(subspaces)

n_max = 40
instances_tested = 0
total_metric_value = 0.0
conjecture_holds = True
counterexample = ""

for depth in range(5, 41):
    for _ in range(6): # Ensure at least 30 instances per seed
        n = random.randint(2, n_max)
        circuit = generate_circuit(depth, n)
        V = generate_vector_space(circuit)
        metric_value = count_affine_subspaces(V)
        total_metric_value += metric_value
        instances_tested += 1

        if depth * depth * math.log(n) < metric_value:
            conjecture_holds = False
            counterexample = f"Depth {depth}, n={n}, Expected 0({depth**2 * math.log(n)}) but

mean_metric_value = total_metric_value / instances_tested
std_metric_value = (sum((x - mean_metric_value) ** 2 for x in range(instances_tested)) / instances_tested) ** 0.5

return {
    "metric_name": "Min Affine Subspaces",
    "metric_value": mean_metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = (sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)

```

```
print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4670d512.py", line 94, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4670d512.py", line 68, in run_trial
    metric_value = count_affine_subspaces(V)
                   ^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4670d512.py", line 54, in count_affine_
    subspaces = find_minimal_affine_subspaces(V)
                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4670d512.py", line 43, in find_minimal_
    m, n = len(V), len(V[0])
             ~^^^
```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Investigate and fix the crash in the test code to continue testing the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15898
2	propose	ollama_remote	glm4:latest	0	0	12759
3	propose	ollama_remote	glm4:latest	0	0	12338
4	preregistration	ollama_remote	glm4:latest	0	0	9200
5	novelty	ollama_remote	glm4:latest	0	0	8956
6	novelty	ollama_remote	glm4:latest	0	0	8703
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12980
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	38065
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22227

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	41052
11	judge	ollama_remote	glm4:latest	0	0	63978

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 246155 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in research/audit/2b2d30ba76b8.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/2b2d30ba76b8.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/2b2d30ba76b8.tar.gz (if generated)